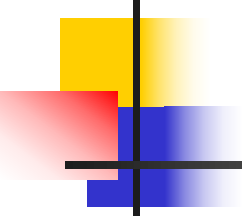


第2章 模型评估与选择



主要内容

- 经验误差与过拟合
- 评估方法
- 性能度量
- 比较检验
- 偏差与方差

2.1 经验误差与过拟合

■ 错误率

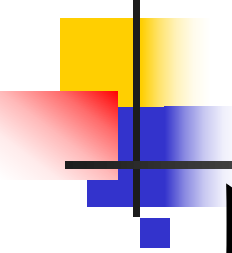
- 分类错误的样本数占样本总数的比例
- $E = a/m$, m 样本数, a 分类错误数

■ 精度

- 1-错误率

■ 误差

- 学习器的实际预测输出与样本的真实输出之间的差异
 - 训练误差(经验误差)
 - 学习器在训练集上的误差
 - 泛化误差
 - 在新样本上的误差



2.1 经验误差与过拟合

■ 目标

- 设计得到泛化误差小的学习器

- 事先不知道新样本

- 只能努力使经验误差最小

- 在训练集上表现好的学习器是不是就好？

■ 过拟合

- 学习器把训练样本学得太多，可能将训练样本不太一般的特性学到，当作一般性质，从而导致泛化性能下降

- 原因

- 学习能力过于强大

- 无法彻底避免，只是缓解，减小其风险

■ 欠拟合

- 对训练样本的一般性质尚未学好

- 学习能力低下

2.1 经验误差与过拟合

树叶训练样本



新样本



过拟合模型分类结果：
→ 不是树叶
(误以为树叶必须有锯齿)



欠拟合模型分类结果：
→ 是树叶
(误以为绿色的都是树叶)



2.1 经验误差与过拟合

■ 模型选择

■ 问题

- 在现实任务中，有多种学习算法、不同的参数配置可供选择，如何选用
- 选择泛化误差最小的那个模型，但无法直接获得泛化误差
 - 无法直接获得泛化误差
- 训练误差由于过拟合现象而不适合作为标准

■ 四个关键

- 评估方法，如何获得测试结果
- 性能度量，如何评估性能优劣
- 比较检验，如何判断实质差别
- 分析说明，如何解释泛化性能

2.2 评估方法

■ 实验测试

- 通过实验测试对学习器的泛化误差进行评估
 - 使用测试集测试学习器对新样本的判别能力
 - 以测试集上的测试误差作为泛化误差的近似

■ 测试集

- 从样本真实分布中独立同分布采样而得
- 尽可能与训练集互斥，即不在训练集中出现或未使用过
- 只有一个包含 m 个样例的数据集 $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$ ，既要训练，又要测试
 - 如何产生出训练集S和测试集T
 - 留出法 (hold-out)
 - 交叉验证法 (cross validation)
 - 自助法 (bootstrap)

2.2 评估方法



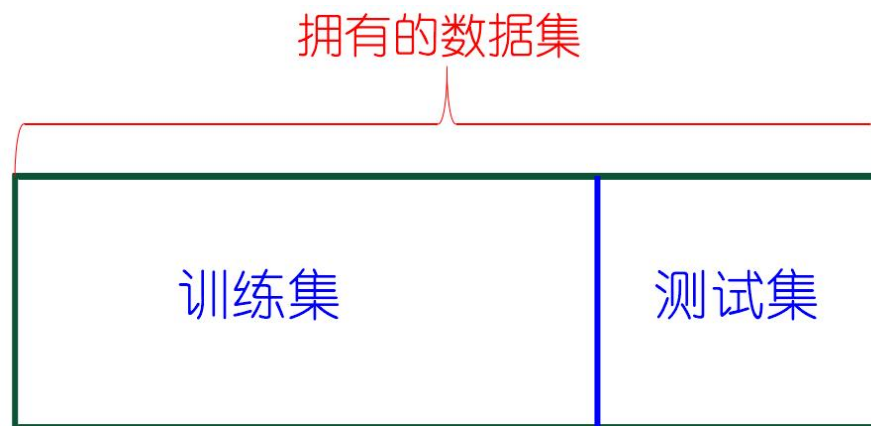
■ 2.2.1 留出法

- 将数据集 D 划分为两个互斥的集合，一个作为训练集 S ，另一个作为测试集 T
 - 训练集、测试集划分的分布
 - 训练集、测试集划分要尽可能保持数据分布的一致性
 - 避免因数据划分引入额外的偏差而对最终结果产生影响
 - 如保留类别比例的采样方式，通常称为分层采样
 - 在给定训练/测试集的样本比例后，仍存在多种划分方式对初始数据集 D 进行分割
 - 如排序划分、随机划分
 - 单次使用留出法得到的估计结果往往不够稳定
 - 采用若干次随机划分、重复进行实验评估后取平均值作为留出法的评估结果

2.2 评估方法

■ 2.2.1 留出法

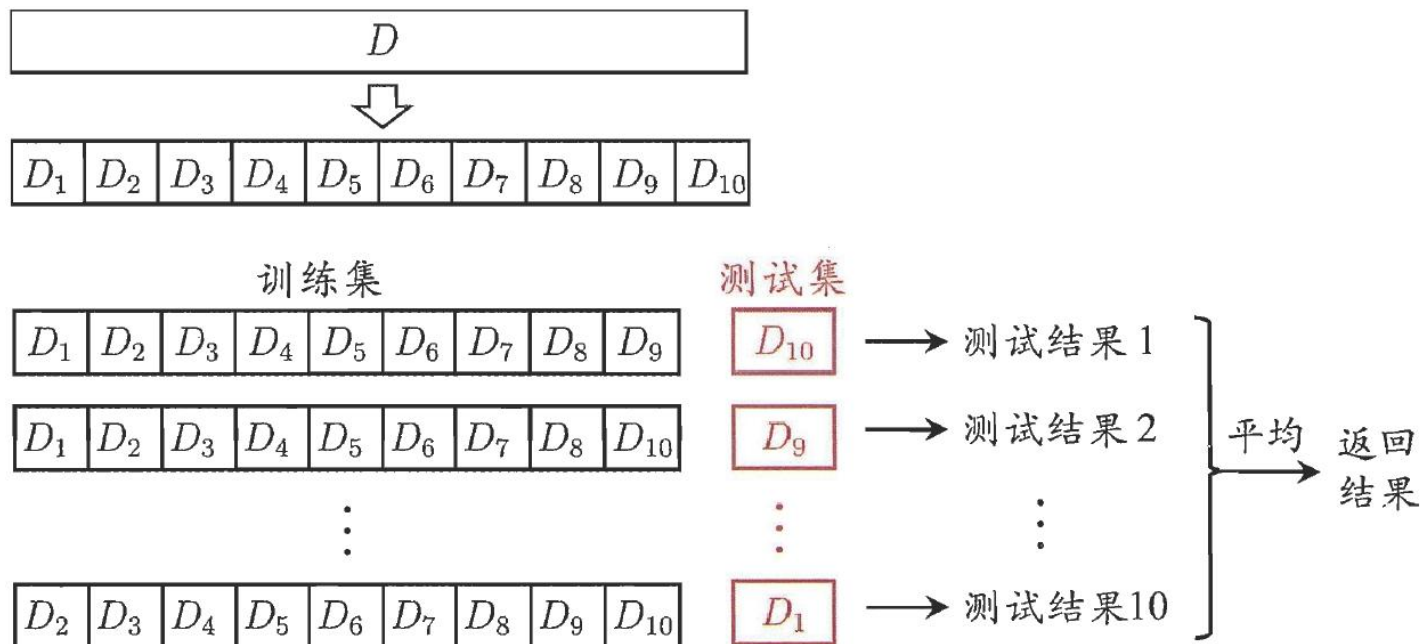
- 将数据集D划分为两个互斥的集合，一个作为训练集S，另一个作为测试集T
 - 划分训练集、测试集的大小
 - 训练集S大测试集T小，训练出的模型更接近于用D训练出的模型，但评估结果可能不够准确
 - 训练集S小测试集T大，被评估的模型与用D训练出的模型有较大差别，降低了评估结果的保真性
 - 没有完美的解决方案，常用2/3-4/5的样本用于训练，剩余用于测试



2.2 评估方法

2.2.2 交叉验证法(k折交叉验证)

- 将数据集D划分为k个大小相似的互斥子集，每个子集尽可能保持数据分布的一致性
 - 每次用k-1个子集的并集作为训练集，余下的子集作为测试集，可得k组训练/测试集
 - 最终返回的是k个测试结果的均值





2.2 评估方法

■ 2.2.2 交叉验证法(k折交叉验证)

- 评估结果的稳定性和保真性在很大程度上取决于k的取值

- 常用10折交叉验证

- k折交叉验证通常要随机使用不同的划分重复p次

- 最终的评估结果是这p次k折交叉验证结果的均值

- 常见10次10折交叉验证

- $k=m$, 称为留一法

- 使用的训练集与初始数据集相比只少了一个样本, 使得绝大多数情况下实际评估的模型与期望评估的用D训练出的模型很相似

- 评估结果往往被认为比较准确

- 缺陷

- 数据集比较大时, 训练计算开销大
 - 留一法的估计结果未必永远比其他评估方法准确, 没有免费的午餐

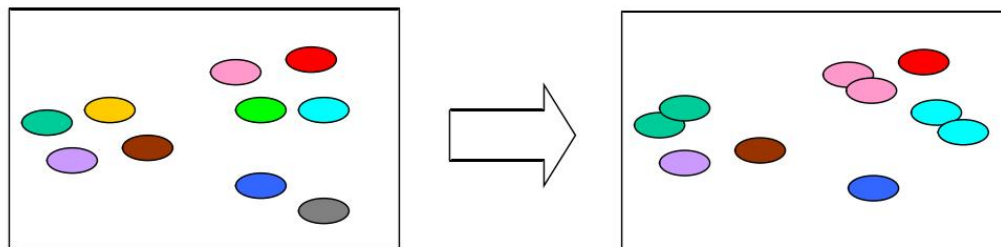
2.2 评估方法

■ 2.2.3 自助法(包外估计)

- 留出法和交叉验证法，保留一部分样本用于测试，实际评估模型所使用训练集比D小
 - 会引入因训练样本规模不同而导致的估计偏差
 - 留一法影响较小，但计算复杂度太高
- 以自助采样法为基础
 - 每次随机从D中挑选一个样本，将其拷贝放入D'
 - 再将该样本放回初始数据集D中，使得该样本在下次采样时仍有可能被采到

基于“自助采样” (bootstrap sampling)

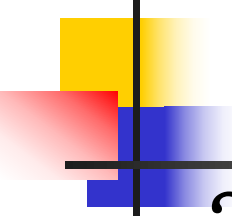
亦称“有放回采样”、“可重复采样”



2.2 评估方法

2.2.3 自助法(包外估计)

- 重复执行 m 次后，得到包含 m 个样本的数据集 D' ，这就是自助采样的结果
 - D 中有一部分样本会在 D' 中多次出现，而另一部分样本不出现
 - m 次采样中始终不被采到的概率是 $(1-1/m)^m$ ，取极限 $\lim_{m \rightarrow \infty} (1-1/m)^m = 1/e \approx 0.368$
- D' 用作训练集， $D-D'$ 用作测试集
 - 训练集与原样本集同规模
 - 数据分布有所改变
- 优缺点
 - 在数据集较小、难有效划分训练、测试集时有用
 - 能产生多个不同的训练集，对集成学习等方法有很大的好处
 - 产生的数据集改变了初始分布，会引入估计偏差



2.2 评估方法

2.2.4 调参与最终模型

■ 调参

- 在进行模型评估与选择时，除了要对学习算法进行选择，还需对算法参数进行设定，称为调参
- 参数
 - 模型的参数，一般由学习确定
 - 算法的参数，一般由人工设定，亦称“超参数”
- 参数调节对最终模型性能有关键性影响
- 设定不同参数先产生若干模型，然后基于某种评估方法选择最佳参数
 - 通常很困难
 - 参数多
 - 实数取值范围
 - 常用方法
 - 设定取值范围
 - 设定变化步长



2.2 评估方法

■ 2.2.4 调参与最终模型

■ 最终模型

- 在模型评估选择过程中，只用了一部分数据训练模型。在模型选择完成后，学习算法和参数配置已定，此时用数据集D重新训练而得到的模型

■ 测试数据与验证数据

- 测试、验证经常混用，但是也有区分
- 测试数据集
 - 实际中用到的数据
 - 测试是对真实结果说的
- 验证集
 - 模型评估与选择中用于评估测试的数据集
 - 验证是对你的期望说的
- 训练数据=训练集+验证集
- 数据集=训练数据+测试数据(集)

2.3 性能度量

■ 性能度量

- 衡量模型泛化能力的评价标准
- 在预测任务中，给定样例集 $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$ ，其中 y_i 是真实标记。要评估学习器 $f(x)$ 的性能，就要把学习器预测结果 $f(x)$ 与真实标记 y 进行比较
- 使用不同的性能度量会导致不同的评判结果
 - 评判模型好坏，不仅取决于算法和数据，还取决于任务需求

2.3 性能度量

■ 均方误差

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m [f(\mathbf{x}_i) - y_i]^2$$

- D 数据集, y_i 示例真实标记, f 学习器, $f(x_i)$ 预测结果
- 一般形式

$$E(f; D) = \int_{\mathbf{x} \sim D} [f(\mathbf{x}) - y]^2 p(\mathbf{x}) d\mathbf{x}$$

- 用于回归任务
- 后面主要介绍分类任务常用的性能度量

2.3 性能度量

■ 2.3.1 错误率与精度

■ 错误率

- 分类错误的样本数占样本总数的比例

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}(f(\mathbf{x}_i) \neq y_i)$$

- 一般形式

$$E(f; D) = \int_{\mathbf{x} \sim D} \mathbb{1}(f(\mathbf{x}) \neq y) p(\mathbf{x}) d\mathbf{x}$$

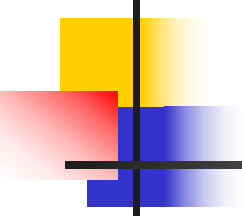
■ 精度

- 分类正确的样本数占样本总数的比例

$$\text{acc}(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}(f(\mathbf{x}_i) = y_i) = 1 - E(f; D)$$

- 一般形式

$$\text{acc}(f; D) = \int_{\mathbf{x} \sim D} \mathbb{1}(f(\mathbf{x}) = y) p(\mathbf{x}) d\mathbf{x} = 1 - E(f; D)$$



2.3 性能度量

■ 2.3.1 错误率与精度

- 既适用于二分类任务，也适用于多分类任务
- 错误率和精度常用，但不能满足所有需求
 - 如问题是“挑出的西瓜中有多少比例是好瓜”，或者“所有好瓜中有多少比例被挑了出来”
 - 错误率显然就不够用，需要使用其他的性能度量

2.3 性能度量

2.3.2 查准率、查全率

■ 混淆矩阵

- 对于二分类，将样例根据真实类别与学习器预测类别划分为真正例、假正例、真反例、假反例四种，令 TP , FP , TN , FN 表示对应的样例数，可构建

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

■ 查准率 $P = \frac{TP}{TP + FP}$

■ 查全率 $R = \frac{TP}{TP + FN}$

2.3 性能度量

2.3.2 查准率、查全率

- 查准率和查全率是一对矛盾的度量
 - 查准率高时，查全率往往偏低
 - 如希望选出的瓜中好瓜比例尽可能高，则可只挑选最有把握的瓜，但这样就难免会漏掉不少好瓜，使得查全率较低
 - 查全率高时，查准率往往偏低
 - 如希望将好瓜尽可能多地选出来，则可通过增加选瓜的数量来实现，如果将所有西瓜都选上，那么所有的好瓜也必然都被选上了，但这样查准率就会较低
 - 有没有办法找到两者相对都高的模型
 - P-R图、平衡点BEP

2.3 性能度量

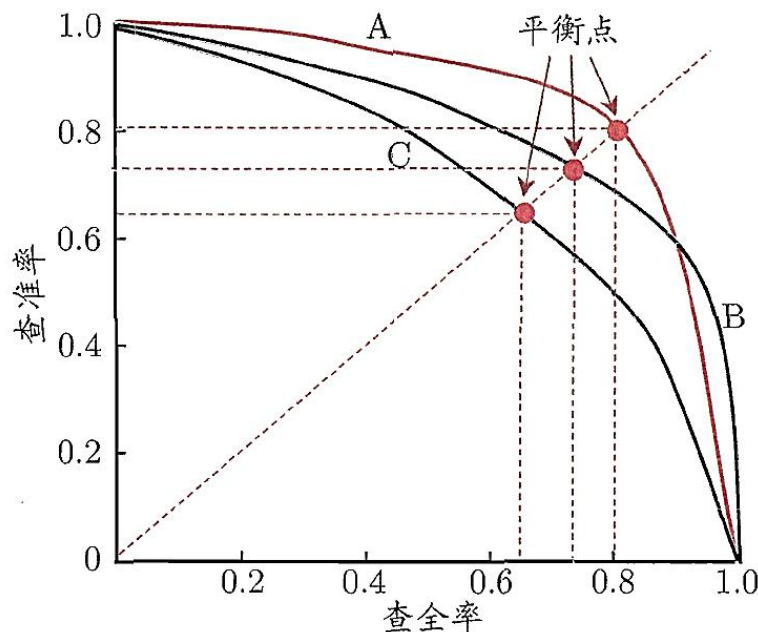
$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$

2.3.2 查准率、查全率

■ P-R图、平衡点BEP

- 根据学习器的预测结果按正例可能性大小对样例进行排序，并逐个把样本作为正例进行预测
- 计算出当前的查全率、查准率作图，简称P-R曲线
- 查准率=查全率时，就是平衡点(BEP)
 - 两者相对双高

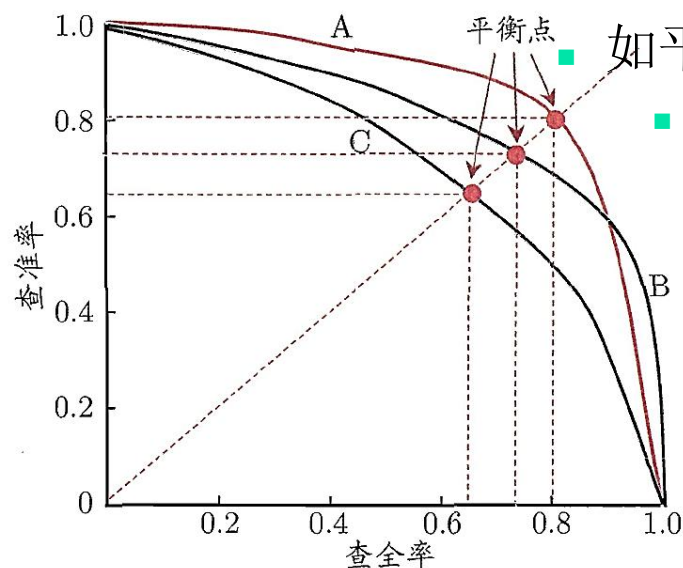


2.3 性能度量

2.3.2 查准率、查全率

■ P-R图、平衡点BEP

- P-R图显示学习器在样本总体上的查全率、查准率
- 一学习器的P-R曲线被另一学习器的曲线完全包住
 - 可断言后者的性能优于前者
- 两学习器的P-R曲线交叉
 - 难以一般性地断言两者孰优孰劣
 - 比较合理的判据是比较P-R曲线下面积的大小
 - 不易估算，设计综合查准率、查全率的性能度量



PR图:

- 学习器 A 优于 学习器 C
- 学习器 B 优于 学习器 C
- 学习器 A ?? 学习器 B

BEP:

- 学习器 A 优于 学习器 B
- 学习器 A 优于 学习器 C
- 学习器 B 优于 学习器 C

2.3 性能度量

■ 2.3.2 查准率、查全率

■ F1度量

- BEP还是过于简化，常用的是F1度量
- 是基于查准率与查全率的调和平均

$$F1 = \frac{2 \times P \times R}{P + R} = \frac{2 \times TP}{\text{样例总数} + TP - TN}$$

- 更重视小值
- 若对查准率、查全率重视程度不同，有不同偏好，定义F1的一般形式，加权调和平均

$$F_{\beta} = \frac{(1 - \beta^2) \times P \times R}{(\beta^2 \times P) + R}$$

- $\beta > 1$ 时查全率有更大影响， $\beta < 1$ 时查准率有更大影响

2.3 性能度量

■ 2.3.2 查准率、查全率

■ 宏与微查准率、查全率、F1

- 有时有多个二分类混淆矩阵，如多次训练测试、或多分类，得到多个混淆矩阵

- 如何估计算法的全局性能

■ 两种方法

- 宏方法：先在各混淆矩阵上分别计算，再计算平均值

$$macro-P = \frac{1}{n} \sum_{i=1}^n P_i \quad macro-R = \frac{1}{n} \sum_{i=1}^n R_i$$

$$macro-F1 = \frac{2 \times macro-P \times macro-R}{macro-P + macro-R}$$

- 微方法：将各混淆矩阵对应元素平均，再对均值计算

$$micro-P = \frac{\overline{TP}}{\overline{TP} + \overline{FP}} \quad micro-R = \frac{\overline{TP}}{\overline{TP} + \overline{FN}}$$

$$micro-F1 = \frac{2 \times micro-P \times micro-R}{micro-P + micro-R}$$



2.3 性能度量

■ 2.3.3 ROC(受试者工作特征)、AUC

- 很多学习器产生实值或概率预测，然后将预测值与分类阈值比较分类
 - 这个实值或概率预测结果的好坏，直接决定了学习器的泛化能力
 - 可将测试样本进行排序
 - 最可能是正例的排在最前面，最不可能是正例的排在最后面
 - 分类就相当于以某个截断点将样本分为两部分
 - 不同的应用任务，采用不同的截断点
 - 重视查准率选靠前位置的截断点，重视查全率选靠后的截断点
 - 需要综合考虑学习器在不同任务下的期望泛化性能的好坏

2.3 性能度量

2.3.3 ROC、AUC

■ ROC(受试者工作特征)曲线

- 表现学习器在不同任务下的“期望泛化性能”

- 纵轴：真正例率TPR；

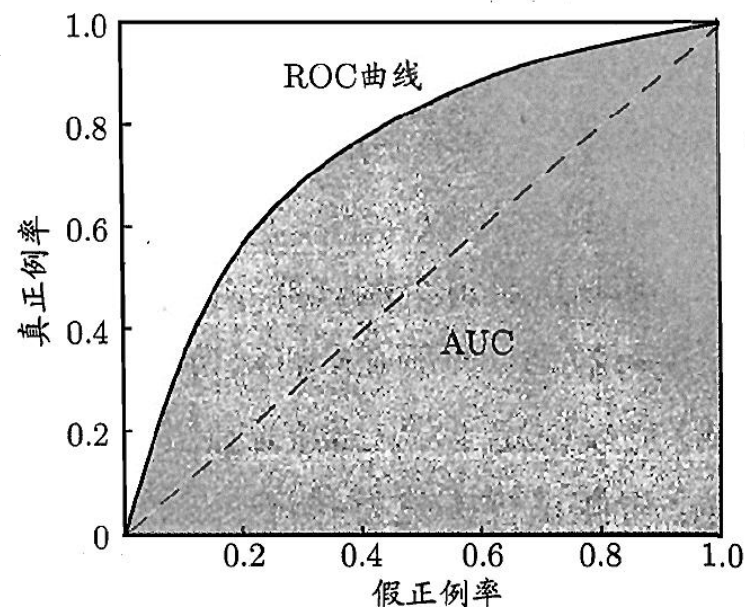
$$TPR = \frac{TP}{TP + FN}$$

- 横轴：假正例率FPR

$$FPR = \frac{FP}{TN + FP}$$

- 对角线：随机猜测模型

- 点(0, 1)：理想模型



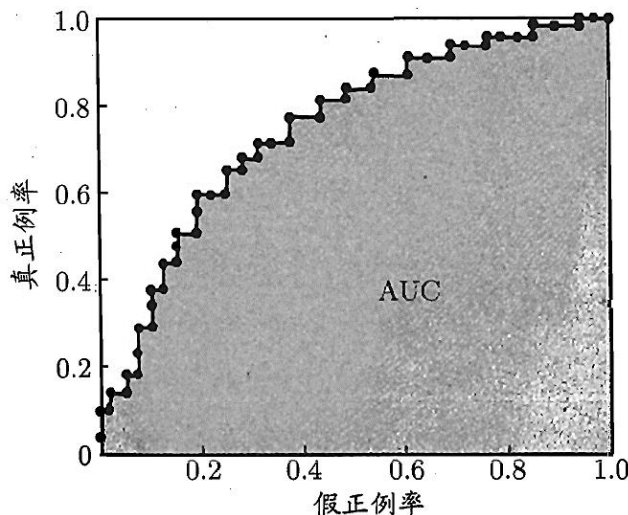
2.3 性能度量

2.3.3 ROC、AUC

■ ROC(受试者工作特征)曲线

■ 现实中测试样例有限, 近似绘制ROC图

- 给定 m^+ 正例和 m^- 反例, 根据学习器预测结果排序
- 然后把分类阈值设为最大, 坐标 $(0, 0)$ 处标记
- 将分类阈值依次设为每个样例的预测值, 计算标记
 - 当前为真正例, 标记点坐标为 $(x, y + 1/m^+)$
 - 当前为假正例, 标记点坐标为 $(x + 1/m^-, y)$
 - (x, y) 为前一个标记点坐标
- 用线段连接相邻点即得



2.3 性能度量

ROC、AUC

ROC (受试者工作特征) 曲线

对学习器比较

- 若一学习器的ROC曲线被另一曲线完全包住，则后者性能优于前者
- 若两个ROC曲线交叉，则难以比较
 - 较为合理的判据是比较ROC曲线下的面积

AUC (Area Under the ROC Curve)

- 估算：ROC曲线由坐标点连接形成

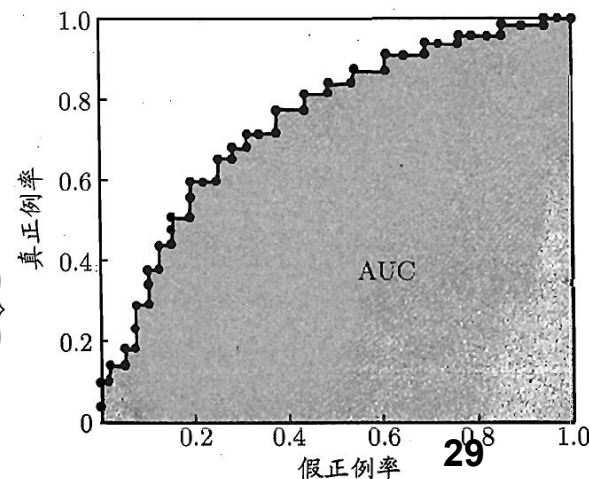
$$AUC = \frac{1}{2} \sum_{i=1}^{m-1} (x_{i+1} - x_i)(y_i + y_{i+1})$$

用排序损失表示

m^+ 个正例， m^- 个反例， D^+ 正例集合， D^- 反例集合

$$l_{rank} = \frac{1}{m^+ m^-} \sum_{x^+ \in D^+} \sum_{x^- \in D^-} \left\{ \hat{0}[f(x^+) < f(x^-)] + \frac{1}{2} \hat{0}[f(x^+) = f(x^-)] \right\}$$

$$AUC = 1 - l_{rank}$$



2.3 性能度量

■ 代价敏感错误率与代价曲线

- 有时不同类型的错误会造成不同的损失，此时可为错误赋予“非均等代价”
- 二分类可根据任务领域知识设定代价矩阵
 - $cost_{ij}$ 表示将第 i 类样本预测为第 j 类样本的代价

真实类别	预测类别	
	第 0 类	第 1 类
第 0 类	0	$cost_{01}$
第 1 类	$cost_{10}$	0

- 代价敏感错误率 (归一化的总体代价)

$$E(f; D; cost) = \frac{1}{m} \left(\sum_{\mathbf{x}_i \in D^+} \mathbb{1}[f(\mathbf{x}_i) \neq y_i] \times cost_{01} + \sum_{\mathbf{x}_i \in D^-} \mathbb{1}[f(\mathbf{x}_i) \neq y_i] \times cost_{10} \right)$$

2.3 性能度量

■ 代价敏感错误率与代价曲线

■ 代价曲线与期望总体代价

- 非均等代价下代价曲线反映学习器期望总体代价
- 横轴：正例概率代价，取值为 $[0, 1]$

$$P(+)\text{cost} = \frac{p \times \text{cost}_{01}}{p \times \text{cost}_{01} + (1 - p) \times \text{cost}_{10}}$$

p , 样例为正例的概率

- 纵轴：归一化代价，取值为 $[0, 1]$

$$\text{cost}_{\text{norm}} = \frac{FNR \times p \times \text{cost}_{01} + FPR \times (1 - p) \times \text{cost}_{10}}{p \times \text{cost}_{01} + (1 - p) \times \text{cost}_{10}}$$

- 其中FPR假正例率， $FNR=1-TPR$ 是假反例率

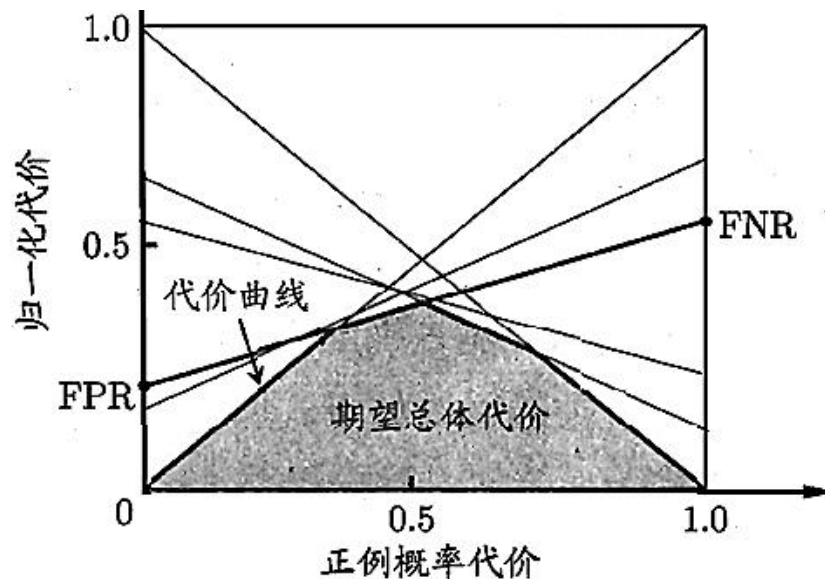
2.3 性能度量

代价敏感错误率与代价曲线

■ 代价曲线与期望总体代价

■ 绘制

- ROC曲线每一点(TPR, FPR)对应代价平面上的一条线段
 - 计算假反利率 $FNR=1-TPR$
- 在代价平面上绘制从(0, FPR)到(1, FNR)的线段
 - 线段下的面积即表示了该条件下的期望总体代价
- 取所有线段的下界，围成的面积即为在所有条件下学习器的期望总体代价



2.4 比较检验

- 取得某种性能度量评估结果后，能否直接比较学习器
 - 比较的是泛化性能，获得的是测试性能
 - 测试性能不等于泛化性能
 - 测试性能随着测试集的变化而变化
 - 大小、样例的不同
 - 很多机器学习算法本身有一定的随机性
 - 多次运行结果会不同
- 统计假设检验 (hypothesis test) 为学习器性能比较提供了重要依据
 - 若在测试集上观察到学习器A比B好，A的泛化性能是否在统计意义上优于B，以及这个结论的把握有多大

2.4 比较检验

■ 假设检验

■ 2.4.1 单个学习器泛化性能的假设进行检验

- 泛化错误率与测试错误率未必相同，但二者接近的可能性比较大，可根据测试错误率估推出
- 根据测试错误率 $\hat{\varepsilon}$ 估计推出泛化错误率 ε 的分布
 - 泛化错误率为 ε 的学习器被测得测试错误率为 $\hat{\varepsilon}$ 的概率

$$P(\hat{\varepsilon}; \varepsilon) = \binom{m}{\hat{\varepsilon} \times m} \varepsilon^{\hat{\varepsilon} \times m} (1 - \varepsilon)^{m - \hat{\varepsilon} \times m}$$

- 给定测试错误率，可得 $P(\hat{\varepsilon}; \varepsilon)$ 在 $\varepsilon = \hat{\varepsilon}$ 时最大，这符合二项(binomial)分布
- 可使用“二项检验”
 - 在 α 的显著度下，假设 $1 - \alpha$ 不能被拒绝，即能以 $\varepsilon \leq \varepsilon_0$ 的置信度认为，学习器的泛化错误率不大于 ε_0 ；否则该假设可被拒绝，即在 α 的显著度下可认为学习器的泛化错误率大于 ε_0

2.4 比较检验

■ 假设检验

■ 2.4.1 单个学习器泛化性能的假设进行检验

- 多次重复留出法或是交叉验证法等进行多次训练测试，会得到多个测试错误率情况下

- 可使用“t检验”

- 对多个测试错误率求平均测试错误率 μ 和方差 σ^2

$$\mu = \frac{1}{k} \sum_{i=1}^k \hat{\varepsilon}_i \quad \sigma^2 = \frac{1}{k-1} \sum_{i=1}^k (\hat{\varepsilon}_i - \mu)^2$$

- 考虑到这k个测试错误率可看作泛化错误率 ε_0 的独立采样，则变量 τ_t 服从自由度为k-1的t分布

$$\tau_t = \frac{\sqrt{k}(\mu - \varepsilon_0)}{\sigma}$$

- 对假设 $\mu = \varepsilon_0$ 和显著度 α ，若平均错误率 μ 与 ε_0 之差位于临界值范围内，则不能拒绝假设 $\mu = \varepsilon_0$ ，其置信度为 $1-\alpha$ ；否则可拒绝该假设，即在该显著度下可认为泛化错误率与 ε_0 有显著不同

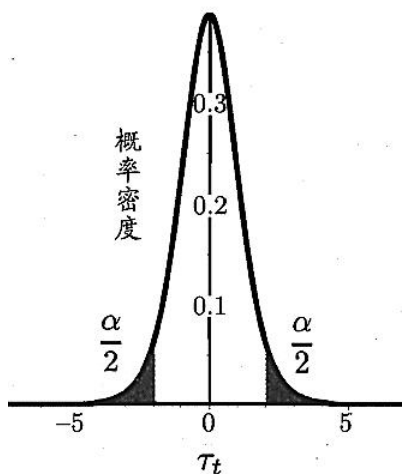


图 2.7 t 分布示意图($k=10$)

2.4 比较检验

假设检验

■ 两学习器比较

■ 2.4.2 交叉验证t检验（基于成对t检验）

■ 使用k折交叉验证

- 设使用k折交叉验证法得到两学习器的测试错误率分别为 $\varepsilon_i^A, \varepsilon_i^B$
- 基本思想是对每对结果求差，若两个学习器性能相同，则差值均值应为零
- 方法：根据差值来对“学习器A与B性能相同”这个假设做t检验，计算出差值 $\varepsilon_i^A - \varepsilon_i^B$ 的均值 μ 和方差 σ^2 ，在显著度下 α ，若变量

$$\tau_t = \frac{\sqrt{k}\mu}{\sigma}$$

小于临界值 $t_{\alpha/2, k-1}$ ，则假设不能被拒绝，即两学习器的性能没有显著差别；否则有显著差别，且平均错误率较小的性能较优

- 不同轮次训练集有重叠，测试错误率不独立，会导致过高估计假设成立的概率

2.4 比较检验

■ 假设检验

■ 两学习器比较

■ 2.4.2 交叉验证t检验（基于成对t检验）

■ 使用5x2交叉验证

- 为缓解上述问题，可采用5x2交叉验证
- 每次2折交叉验证前数据随机打乱，5次划分不重复
- 第i次2折交叉验证将产生两对测试错误率，分别求第1折上的差 Δ_i^1 和第2折上的差 Δ_i^2 ，计算两个结果的平均值 $\mu = 0.5(\Delta_i^1 + \Delta_i^2)$
- 对每次2折实验的结果都计算出其方差

$$\sigma_i^2 = \left(\Delta_i^1 - \frac{\Delta_i^1 + \Delta_i^2}{2} \right)^2 + \left(\Delta_i^2 - \frac{\Delta_i^1 + \Delta_i^2}{2} \right)^2$$

- 变量 τ_t 服从自由度为5的t分布

$$\tau_t = \frac{\mu}{\sqrt{0.2 \sum_{i=1}^5 \sigma_i^2}}$$

- 给定显著度 α ，可判别是否有显著差别

2.4 比较检验

假设检验

■ 两学习器比较

■ 2.4.3 McNemar检验

- 对于二分类问题，可基于留出法列联表
 - 获得两者都正确、都错误、一正确另一错误的样本数
- 若两学习器性能相同，则 $e_{01} = e_{10}$
- 变量 $|e_{01} - e_{10}|$ 服从正态分布，而 τ 服从自由度为1的 χ^2 分布
$$\tau_{\chi^2} = \frac{(|e_{01} - e_{02}| - 1)^2}{e_{01} + e_{02}}$$
- 给定显著度 α ，可判别是否有显著差别
 - 有显著差别且平均错误率较小的学习器性能较优

表 2.4 两学习器分类差别列联表

算法 B	算法 A	
	正确	错误
正确	e_{00}	e_{01}
错误	e_{10}	e_{11}

2.4 比较检验

假设检验

■ 多学习器比较

■ 两两比较

- 每个数据集上分别列出两两比较的结果
- 可使用前述方法

■ 2.4.4 Friedman检验+Nemenyi后续检验(基于排序)

■ Friedman检验

- N个数据集对k个算法进行比较
- 用留出法或交叉验证法得到每个算法在每个数据集上的测试结果
- 在每个数据集根据测试由好到坏排序，赋序值1、2...
- 若算法的测试性能相同，则平分序值
- 求第i个算法的平均序值 $\bar{r}_i$ ，其服从正态分布
 - 均值 $(k+1)/2$ ，方差 $(k^2-1)/12$

表 2.5 算法比较序值表

数据集	算法 A	算法 B	算法 C
D_1	1	2	3
D_2	1	2.5	2.5
D_3	1	2	3
D_4	1	2	3
平均序值	1	2.125	2.875

2.4 比较检验

■ 假设检验

■ 多学习器比较

■ 2.4.4 Friedman检验+Nemenyi后续检验(基于排序)

■ Friedman检验

■ 构造变量

$$\tau_{\chi^2} = \frac{12N}{k(k+1)} \left(\sum_{i=1}^k r_i^2 - \frac{k(k+1)^2}{4} \right)$$

- 在k和N都较大时，服从自由度为k-1的 χ^2 分布

$$\tau_F = \frac{(N-1)\tau_{\chi^2}}{N(k-1) - \tau_{\chi^2}}$$

- 服从自由度为k-1和(k-1)(N-1)的F分布
- 若所有算法的性能相同假设被拒绝，则说明算法的性能显著不同
 - 需进行后续检验进一步区分各算法

2.4 比较检验

假设检验

多学习器比较

2.4.4 Friedman检验+Nemenyi后续检验

Nemenyi检验

- 选择置信度计算平均序值差别的临界值域

$$CD = q_{\alpha} \sqrt{\frac{k(k+1)}{6N}}$$

- 若两个算法的平均序值差超出临界阈值CD，则以置信度拒绝“两个算法性能相同”假设，两算法有显著差异，否则接受

Friedman检验图

- 上述检验比较可以直观地用Friedman检验图显示
- 横轴为平均序值
 - 每个算法圆点为其平均序值，线段为临界阈值
- 若两个算法有交叠，则说明没有显著差别
- 否则有显著差别，算法A显著优于算法C

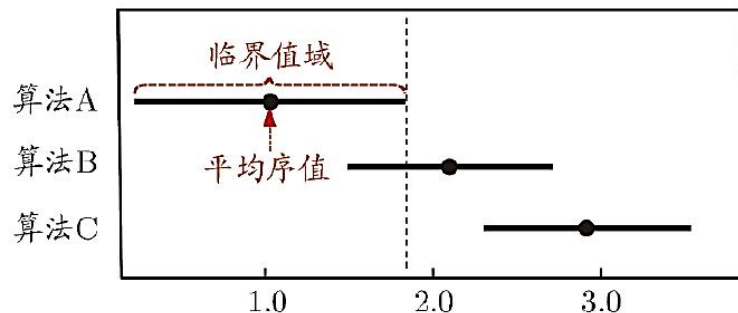


图 2.8 Friedman 检验图

2.5 偏差与方差

- 算法误差从何而来，误差包含哪些因素
 - 解释学习算法泛化性能的工具，偏差-方差
 - 符号
 - y_D 数据集中标记， y 真实标记， $f(\mathbf{x}; D)$ 学得模型

- 对于回归分析

- 期望预测

$$\bar{f}(\mathbf{x}) = E_D[f(\mathbf{x}; D)]$$

- 不同训练集产生的方差

$$\text{var}(\mathbf{x}) = E_D[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2]$$

- 噪声

$$\varepsilon^2 = E_D[(y_D - y)^2]$$

- 期望输出与真实标记的差别称为偏差

$$\text{bias}^2(\mathbf{x}) = [\bar{f}(\mathbf{x}) - y]^2$$

2.5 偏差与方差

■ 对于回归分析

■ 期望泛化误差

■ 假定噪声期望为零

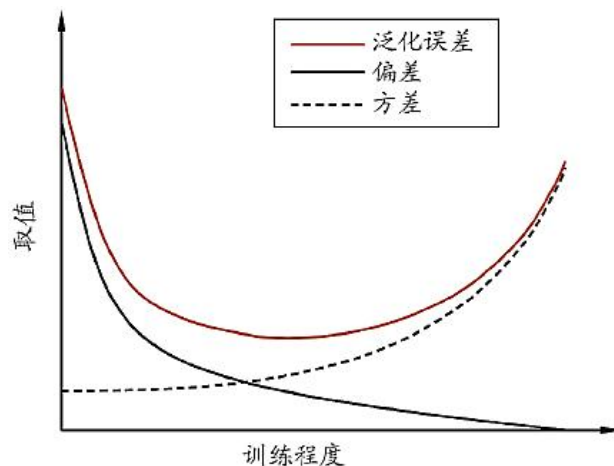
$$\begin{aligned} E(f; D) &= E_D[(f(\mathbf{x}; D) - y_D)^2] \\ &= E_D[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}) + \bar{f}(\mathbf{x}) + y_D)^2] \\ &= \text{bias}^2(\mathbf{x}) + \text{var}(\mathbf{x}) + \varepsilon^2 \end{aligned}$$

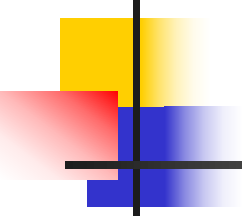
- 偏差 $\text{bias}^2(\mathbf{x}) = [\bar{f}(\mathbf{x}) - y]^2$
 - 期望预测与真实结果的差别，反映学习算法拟合能力
- 方差 $\text{var}(\mathbf{x}) = E_D[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2]$
 - 同样大小训练集变动导致的学习性能变化，即数据扰动造成的影响
- 噪声 $\varepsilon^2 = E_D[(y_D - y)^2]$
 - 表达在当前任务任何学习算法所能达到的期望泛化误差下界，即学习问题的难度
- 学习算法的泛化性能是由学习的能力、数据的充分性以及学习任务本身的难度共同决定

2.5 偏差与方差

■ 偏差-方差窘境

- 给定学习任务，为取得好的泛化性能，需使偏差较小，即能够充分拟合数据，并且使方差较小，即使得数据扰动产生的影响小
- 偏差-方差存在冲突，称为偏差-方差窘境
 - 训练不足，学习器拟合能力不强，训练数据扰动不足以使学习器显著变化，偏差主导
 - 训练程度加深，学习器拟合逐渐增强，训练数据扰动渐渐被学习器学到，方差逐渐主导
 - 训练充足，学习器的拟合很强，训练数据发生轻微扰动导致学习器显著变化，过拟合，方差主导





作业
